Screenshots Module 3 Portfolio Milestone – Option 2

Original Images

Alex.png

the_rock.png

Result Image

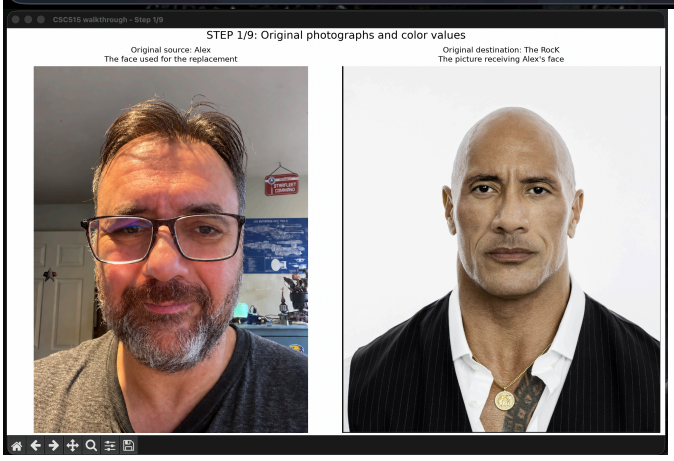
alex_on_the_rock.png

Please also see the direct screenshots from the program execution.

See next page

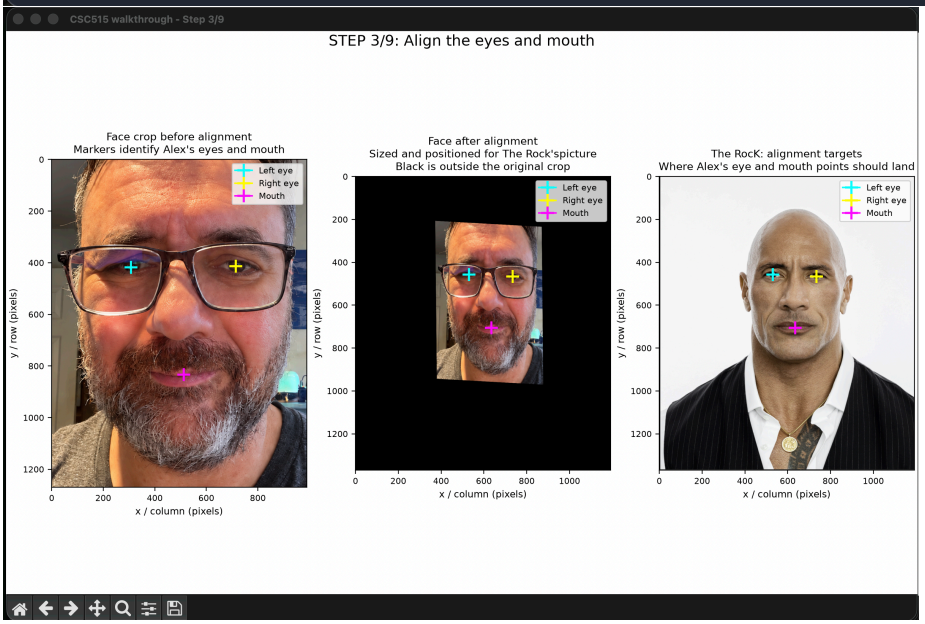
Screenshots

```
CSC515-CTAs — Python Portfolio-Milestone-Module-3-Option-2/Module3_Portfolio_Option2.py — 142×67

alex.omegapy@MacStudio CSC515-CTAs % .venv/bin/python Portfolio-Milestone-Module-3-Option-2/Module3_Portfolio_Option2.py

================================================================================
CSC515 - Portfolio Milestone Module 3
Option #2: Blending Images in OpenCV
================================================================================
The program places Alex's face into The Rock'sphotograph.

Cropping keeps the part of Alex's photograph that contains the face. An affine transform then maps
Alex's eyes and mouth onto The Rock'smatching points, adjusting the face's size, angle, and
position. These coordinates were chosen manually for the supplied photographs.

A blending mask tells the program how much of Alex and The Rock to use at each position. The center
uses Alex's face, and the edge gradually mixes with The Rock'sphotograph. Separate skin masks select
small forehead and cheek patches to compare their colors. Color matching moves Alex's skin colors
toward The RocK's, with less change near the glasses and beard.

Both methods use the corrected face and The Rock'soriginal photograph. Alpha blending mixes their
colors using one weight per pixel. Normal cloning uses changes between neighboring pixels and The
Rock'scolors at the boundary. Keeping the original blend provides a starting result for the
comparison.
Saved full-resolution result:
/Volumes/P-SSD/Omegapy/CSU-projects/CSC515-CTAs/Portfolio-Milestone-Module-3-Option-2/mod_images/alex_on_the_rock.png
Saved comparison:
/Volumes/P-SSD/Omegapy/CSU-projects/CSC515-CTAs/Portfolio-Milestone-Module-3-Option-2/mod_images/face_blending_comparison.png
Opening step 1/9...

================================================================================
STEP 1/9: Original photographs and color values
================================================================================
Load the two photographs and identify their pixel arrays

alex_original_image holds Alex's photograph, which supplies the replacement face.
the_rock_original_image holds The Rock'sphotograph, which receives that face. Loading either file
creates an array in memory and leaves the original file unchanged.

  Code in main():
    alex_original_image = load_bgr_image(ALEX_IMAGE_PATH)
    the_rock_original_image = load_bgr_image(THE_ROCK_IMAGE_PATH)
  Code in show_bgr_image():
    cv2.cvtColor(bgr_image, cv2.COLOR_BGR2RGB)

An image array is a grid of pixels. Each pixel stores blue, green, and red values in BGR order.
image[row, column] selects one pixel; its three entries describe color, not depth. OpenCV uses BGR,
while Matplotlib expects RGB. The display conversion puts those same values in the order Matplotlib
needs.

The color-order matrix for one pixel:
  [R_display]   [0  0  1] [B_image]
  [G_display] = [0  1  0] [G_image]
  [B_display]   [1  0  0] [R_image]

Each 1 selects a value for the display. This matrix describes BGR-to-RGB reordering; the program
uses cvtColor() to do it. The pixel stays at the same row and column. The processing arrays keep
their BGR order.

Reading the pictures:
Left: Alex's original photograph. Right: The Rock'soriginal photograph. Both pictures show the
inputs before cropping, color matching, or blending.

Array shapes: alex_original_image (2048, 1536, 3); the_rock_original_image (1370, 1192, 3). Both use
uint8 BGR values from 0 to 255.
Saved teaching view:
/Volumes/P-SSD/Omegapy/CSU-projects/CSC515-CTAs/Portfolio-Milestone-Module-3-Option-2/mod_images/diagnostics/walkthrough/stage_01.png

Enter C to continue or Q to quit, then press Enter.
```

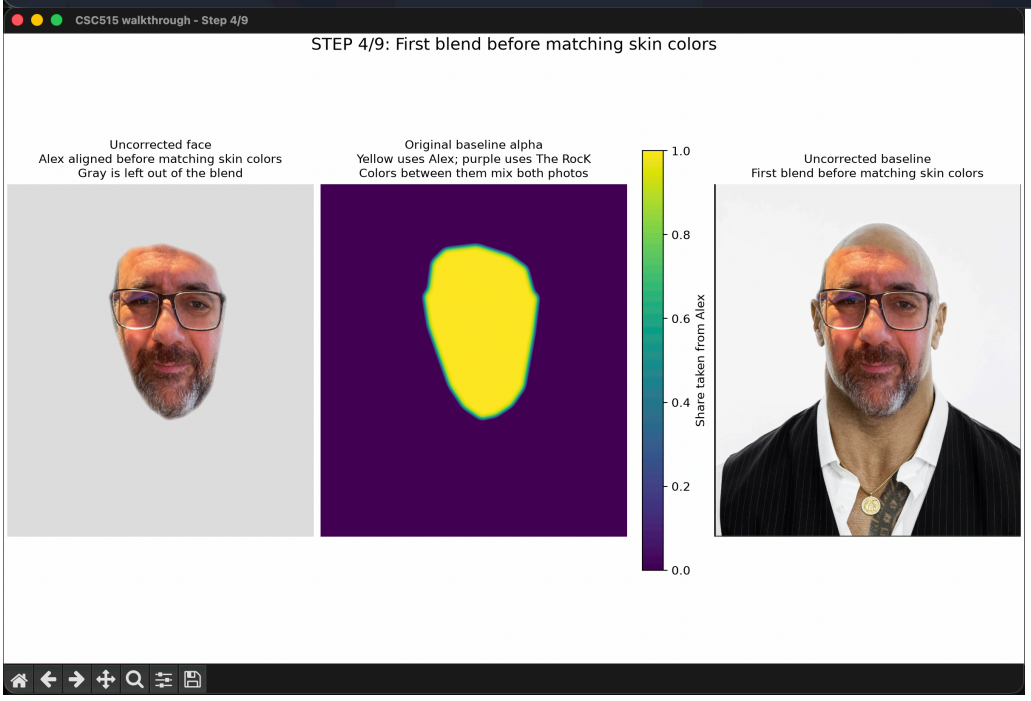


```
Enter C to continue or Q to quit, then press Enter.
c
Opening step 2/9...

================================================================================
STEP 2/9: Crop the source face
================================================================================
Keep Alex's face and measure its marked points from the crop's corner

Cropping selects the rectangle around Alex's face. ALEX_FACE_CROP_BOUNDS stores left, top, right,
and bottom. The slice below keeps those rows and columns, including all three color channels. It
selects pixels without resizing them.

  Code in align_alex_face_and_build_baseline_mask():
    alex_crop_left, alex_crop_top, alex_crop_right, alex_crop_bottom = ALEX_FACE_CROP_BOUNDS
    alex_face_crop = alex_original_image[alex_crop_top:alex_crop_bottom, alex_crop_left:alex_crop_right]
  Code in get_alex_to_the_rock_transform():
    alex_crop_alignment_points = ALEX_FACE_ALIGNMENT_POINTS - np.float32(ALEX_FACE_CROP_BOUNDS[:2])

Array indexing uses rows before columns, so top:bottom comes before left:right. The right and bottom
limits are excluded. The marked eyes and mouth originally use coordinates from the full photograph.
After cropping, the upper-left corner of the crop becomes the new coordinate starting point.

The coordinate change for each marked point:
  [x_in_crop]   [x_in_Alex]   [alex_crop_left]
  [y_in_crop] = [y_in_Alex] - [alex_crop_top ]

  ALEX_FACE_ALIGNMENT_POINTS - crop corner -> alex_crop_alignment_points

ALEX_FACE_CROP_BOUNDS[:2] selects left and top. Subtracting them gives the same feature a new
address inside the crop. NumPy applies that subtraction to each of the three [x, y] rows in
ALEX_FACE_ALIGNMENT_POINTS.

Example: ALEX_FACE_ALIGNMENT_POINTS[0] marks the viewer's left eye in Alex's full photo.
alex_crop_alignment_points[0] marks that same eye inside the crop.

Reading the pictures:
Left: Alex's photograph with the crop rectangle. Right: the extracted face. Matching marker colors
identify the same eyes and mouth in both pictures. The markers are display annotations; they are not
painted into the face pixels.

Array shape: alex_original_image (2048, 1536, 3) -> alex_face_crop (1270, 990, 3). The crop keeps
uint8 BGR pixels.
Saved teaching view:
/Volumes/P-SSD/Omegapy/CSU-projects/CSC515-CTAs/Portfolio-Milestone-Module-3-Option-2/mod_images/diagnostics/walkthrough/stage_02.png

Enter C to continue or Q to quit, then press Enter.
```

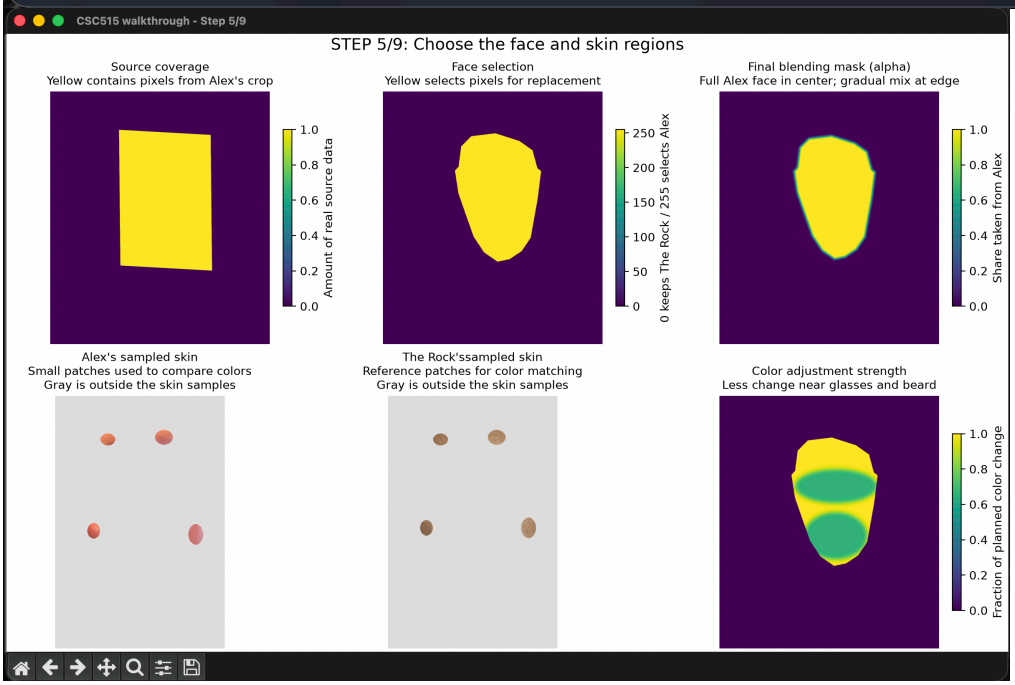
```
Enter C to continue or Q to quit, then press Enter.
c
Opening step 3/9...

================================================================================================================
STEP 3/9: Align the eyes and mouth
================================================================================================================
Fit Alex's eyes and mouth to their positions in The Rock'spicture

The faces have different sizes and positions. The program uses the two eye centers and the mouth
center as matching points. alex_crop_alignment_points holds Alex's positions inside the crop.
THE_ROCK_FACE_ALIGNMENT_POINTS holds the positions where those features should land in The
Rock'sphotograph.

  Code in get_alex_to_the_rock_transform():
    cv2.getAffineTransform(alex_crop_alignment_points, THE_ROCK_FACE_ALIGNMENT_POINTS)
  Code in align_alex_face_and_build_baseline_mask():
    alex_to_the_rock_matrix = get_alex_to_the_rock_transform()
    aligned_alex_face = cv2.warpAffine(
        alex_face_crop, alex_to_the_rock_matrix, (the_rock_width, the_rock_height),
        flags=cv2.INTER_LINEAR, borderMode=cv2.BORDER_CONSTANT,
        borderValue=0)

getAffineTransform() calculates a 2-by-3 matrix from the three point pairs. This affine transform
can resize, turn, move, stretch, or slant the crop. warpAffine() applies that mapping to the face
pixels. INTER_LINEAR estimates colors when a mapped position falls between existing pixels. The face
mask receives the same transform so it stays lined up with the face.

Calculate the pixel's position in The Rock's image:
  Alex's cropped face: (x_in_crop, y_in_crop)
  The Rock's image: (x_on_rock, y_on_rock)

  x_on_rock = m00 * x_in_crop + m01 * y_in_crop + m02
  y_on_rock = m10 * x_in_crop + m11 * y_in_crop + m12

x is the horizontal position; y is the vertical position, in pixels. The first equation calculates
the new x position, and the second calculates y. m00 through m12 label the six entries in
alex_to_the_rock_matrix. The first digit identifies the row and the second identifies the column,
starting at zero. The first two columns control size, angle, and shear. m02 shifts the horizontal
position; m12 shifts the vertical position. These equations expand matrix multiplication by
[x_in_crop, y_in_crop, 1]. The final 1 makes the shifts contribute as m02 * 1 and m12 * 1. This
changes pixel positions; it does not rearrange BGR channels.

Reading the pictures:
Left: Alex's crop with its marked points. Middle: the aligned face. Right: The Rock'starget points.
Matching marker colors identify the same eye or mouth. Black around the aligned face is empty space
outside Alex's crop; the blending mask later excludes it.

Array shape: alex_face_crop (1270, 990, 3) -> aligned_alex_face (1370, 1192, 3). OpenCV receives the
output size as (width, height).
Saved teaching view:
/Volumes/P-SSD/Omegapy/CSU-projects/CSC515-CTAs/Portfolio-Milestone-Module-3-Option-2/mod_images/diagnostics/walkthrough/stage_03.png

Enter C to continue or Q to quit, then press Enter.
```



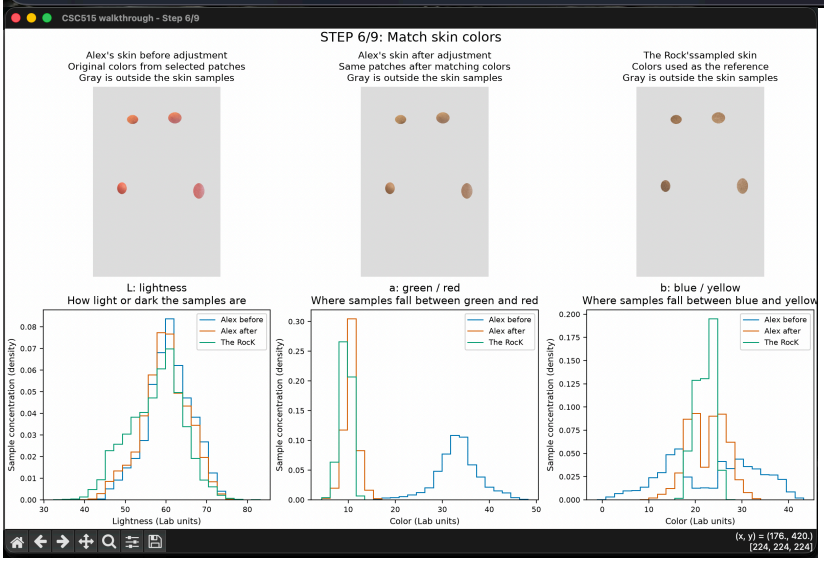
```
Enter C to continue or Q to quit, then press Enter.
c
Opening step 4/9...

================================================================================================
STEP 4/9: First blend before matching skin colors
================================================================================================
Make the first blend before adjusting Alex's skin colors

The baseline is the first blended picture, kept as a starting result for comparison. It combines the
aligned face with The Rock'sphotograph before skin-color matching. Later results start from the same
original photographs.

  Code in align_alex_face_and_build_baseline_mask():
    alex_softened_baseline_mask = cv2.GaussianBlur(
        alex_aligned_baseline_mask, ALEX_BASELINE_MASK_KERNEL, sigmaX=ALEX_BASELINE_MASK_SIGMA)
    alex_baseline_alpha = np.clip(2.0 * alex_softened_baseline_mask - 1.0, 0.0, 1.0)
  Code in create_alex_on_the_rock_results():
    aligned_alex_face, alex_baseline_alpha = align_alex_face_and_build_baseline_mask(alex_original_image, the_rock_original_image)
    alex_on_the_rock_baseline = alpha_blend_foreground_with_background(aligned_alex_face, the_rock_original_image, alex_baseline_alpha)

A mask is a separate image that selects part of a photograph. Here it follows Alex's face and leaves
out the room. A sharp selection could leave a pasted edge. GaussianBlur() averages nearby mask
values to make a gradual transition. ALEX_BASELINE_MASK_KERNEL sets the neighborhood size;
ALEX_BASELINE_MASK_SIGMA controls how widely nearby values contribute. The filter smooths the mask
while the face keeps its detail.

How the mask becomes a blending weight:
  alex_aligned_baseline_mask -> GaussianBlur -> alex_softened_baseline_mask -> alex_baseline_alpha
                                                                                      |
                                                                returned as alex_baseline_alpha

  alex_baseline_alpha = 0          0 < alex_baseline_alpha < 1          alex_baseline_alpha = 1
  The Rock           mix both photos        Alex

Rescaling the softened mask fades the selection inward. The program also sets alex_baseline_alpha to
zero where alex_aligned_baseline_mask is below 1, excluding partially covered edge pixels. This
keeps the black space outside Alex's crop out of the blend. An opaque interior means
alex_baseline_alpha is exactly 1 there.

Reading the pictures:
Left: the aligned face before color matching, shown on gray. Middle: the first blending mask. Yellow
uses Alex, purple keeps The RocK, and the colors between them mix both photographs. Right: the first
blend made with that mask. Gray is only the face preview's background.

Array shapes: face and baseline (1370, 1192, 3); alex_baseline_alpha (1370, 1192). The mask holds
one float32 weight per pixel.
Saved teaching view:
/Volumes/P-SSD/Omegapy/CSU-projects/CSC515-CTAs/Portfolio-Milestone-Module-3-Option-2/mod_images/diagnostics/walkthrough/stage_04.png

Enter C to continue or Q to quit, then press Enter.
```



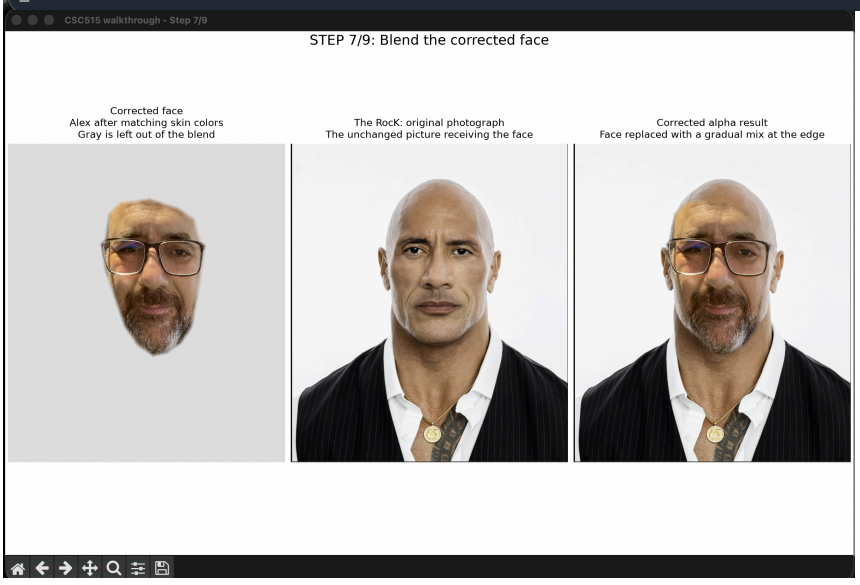
```
Enter C to continue or Q to quit, then press Enter.
c
Opening step 5/9...

================================================================================
STEP 5/9: Choose the face and skin regions
================================================================================
Separate the face selection, skin samples, and amount of color change

These maps have different jobs. alex_crop_coverage records how much of Alex's crop contributes at
each aligned position. alex_face_selection_mask selects the face with 0 or 255. alex_blend_alpha
controls how much of that face appears in the blend. Its center uses Alex fully, while its edge
gradually mixes with The RocK.

  Code in prepare_alex_and_the_rock_masks():
    alex_blend_alpha = create_alex_face_blend_alpha(alex_face_selection_mask, alex_crop_coverage, ALEX_FACE_MASK_FEATHER_PIXELS)
    alex_skin_sample_mask = create_ellipse_mask(alex_blend_alpha.shape, ALEX_SKIN_SAMPLE_PATCHES)
    the_rock_skin_sample_mask = create_ellipse_mask(alex_blend_alpha.shape, THE_ROCK_SKIN_SAMPLE_PATCHES)
  Code in create_alex_face_blend_alpha():
    alex_edge_fraction = np.clip((alex_face_edge_distance - 1.0) / alex_feather_width, 0.0, 1.0)
    alex_blend_alpha = (alex_edge_fraction * alex_edge_fraction * (3.0 - 2.0 * alex_edge_fraction)).astype(np.float32)

The skin masks select small forehead and cheek patches for color measurements. Glasses, lips, and
beard are excluded from those samples because they do not represent skin color. They still appear in
the replacement. alex_color_strength is another map: it reduces the color adjustment near the
glasses and beard.

One position, three different decisions:
  alex_face_selection_mask[y, x]       -> select the replacement face: 0 or 255
  alex_blend_alpha[y, x]      -> decide Alex's share in the blend: 0 to 1
  alex_color_strength[y, x] -> control the planned color adjustment: 0 to 1

  alex_face_edge_distance from the mask's outside -> alex_edge_fraction -> alex_blend_alpha

distanceTransform() measures distance to the mask's outside. Subtracting 1 starts the outermost
selected pixels at zero. Dividing by ALEX_FACE_MASK_FEATHER_PIXELS sets the transition width. The
alex_edge_fraction*alex_edge_fraction*(3-2*alex_edge_fraction) curve starts and ends gently,
avoiding a sudden change in the blending weight.

Example: A position can have alex_blend_alpha=1 and alex_color_strength below 1. The blend uses
Alex's face fully there, while the color adjustment remains reduced.

Reading the pictures:
Top row, left to right: real crop coverage, face selection, and blending weights. Bottom row: Alex's
sampled skin, The Rock'ssampled skin, and color adjustment strength. Gray around the samples makes
the small patches easier to see; it is not included in the measurements.

Each map has shape (1370, 1192). Binary masks use uint8 values 0 and 255; alex_crop_coverage,
alex_blend_alpha, and alex_color_strength use float32 fractions.
Saved teaching view:
/Volumes/P-SSD/Omegapy/CSU-projects/CSC515-CTAs/Portfolio-Milestone-Module-3-Option-2/mod_images/diagnostics/walkthrough/stage_05.png

Enter C to continue or Q to quit, then press Enter.
```



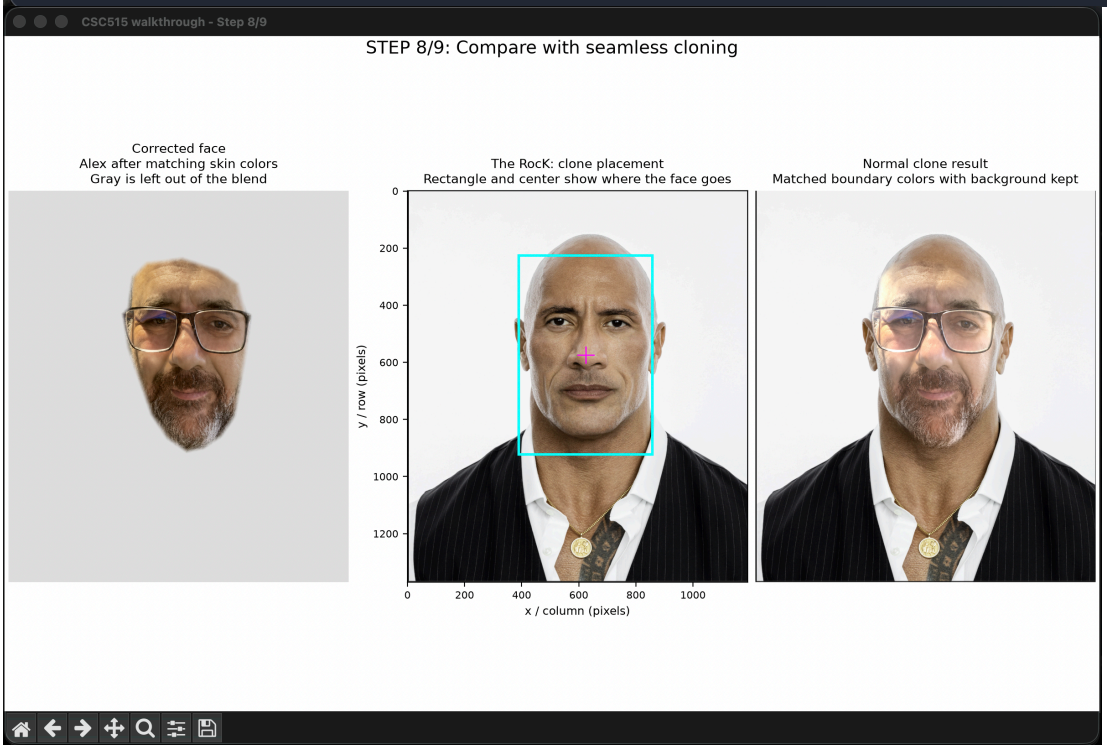
```
Enter C to continue or Q to quit, then press Enter.
c
Opening step 6/9...

================================================================================
STEP 6/9: Match skin colors
================================================================================
Bring Alex's sampled skin colors closer to The Rock'scolors

The photographs have different lighting and skin colors. The program measures the selected patches
and adjusts Alex's colors toward The RocK's. CIE Lab separates lightness, L, from the green/red
direction, a, and the blue/yellow direction, b. This allows a stronger color adjustment with less
change to lightness. Dividing BGR values by 255 supplies the 0-to-1 input range needed for OpenCV's
floating-point Lab conversion.

  Code in match_alex_skin_to_the_rock():
    alex_lab = cv2.cvtColor(
        aligned_alex_face.astype(np.float32) / 255.0, cv2.COLOR_BGR2Lab)
    the_rock_lab = cv2.cvtColor(
        the_rock_original_image.astype(np.float32) / 255.0, cv2.COLOR_BGR2Lab)
    alex_skin_values = alex_lab[alex_skin_sample_mask > 0]
    the_rock_skin_values = the_rock_lab[the_rock_skin_sample_mask > 0]

alex_skin_mean and the_rock_skin_mean each contain three averages: one for L, a, and b. Their
standard deviations describe how much the samples vary. alex_to_the_rock_lab_gain adjusts that
spread. The program limits gain and the proposed shift, then applies the channel strengths and the
correction map. The stronger adjustment for a reduces Alex's red tint. Allowing a narrower color
spread also keeps less-red areas from being pushed too far toward green.

The adjustment, applied separately to L, a, and b:
  alex_to_the_rock_lab_gain = the_rock_skin_std / alex_skin_std, limited by ALEX_TO_THE_ROCK_LAB_GAIN_BOUNDS
  alex_target_lab = (alex_lab - alex_skin_mean) * alex_to_the_rock_lab_gain + the_rock_skin_mean
  alex_lab_shift = alex_target_lab - alex_lab, limited by MAX_ALEX_LAB_SHIFT
  alex_corrected_lab = alex_lab + alex_color_strength * alex_channel_strength * alex_lab_shift

Subtracting Alex's mean centers his sampled colors around zero. Multiplying by
alex_to_the_rock_lab_gain changes their spread, and adding The Rock'smean moves the center toward
his skin colors. A nearly uniform sample keeps alex_to_the_rock_lab_gain at 1 to avoid dividing by a
very small spread. alex_channel_strength holds the three channel amounts. alex_color_strength
supplies a separate amount at each pixel. The displayed equation summarizes that operation; the code
uses alex_color_strength[..., None] so its value applies to all three channels. The adjustment
changes colors without blurring the face.

Example: alex_lab[alex_skin_sample_mask > 0] keeps one [L, a, b] row per selected skin pixel.
mean(axis=0) averages down those rows, keeping one mean for each of the three channels.

Reading the pictures:
Top row: Alex's sampled colors before adjustment, the same patches after adjustment, and The
Rock'sreference samples. The graphs below compare L, a, and b. A higher line means more sampled
colors are concentrated near that value. Each graph uses shared bins and density, so the area under
each distribution is 1 even when the sample counts differ. The graphs show the adjustment; drawing
them does not change the image.

Face shape stays (1370, 1192, 3). uint8 BGR -> float32 Lab -> uint8 BGR. Skin samples have shape
(selected pixel count, 3).
Saved teaching view:
/Volumes/P-SSD/Omegapy/CSU-projects/CSC515-CTAs/Portfolio-Milestone-Module-3-Option-2/mod_images/diagnostics/walkthrough/stage_06.png

Enter C to continue or Q to quit, then press Enter.
```



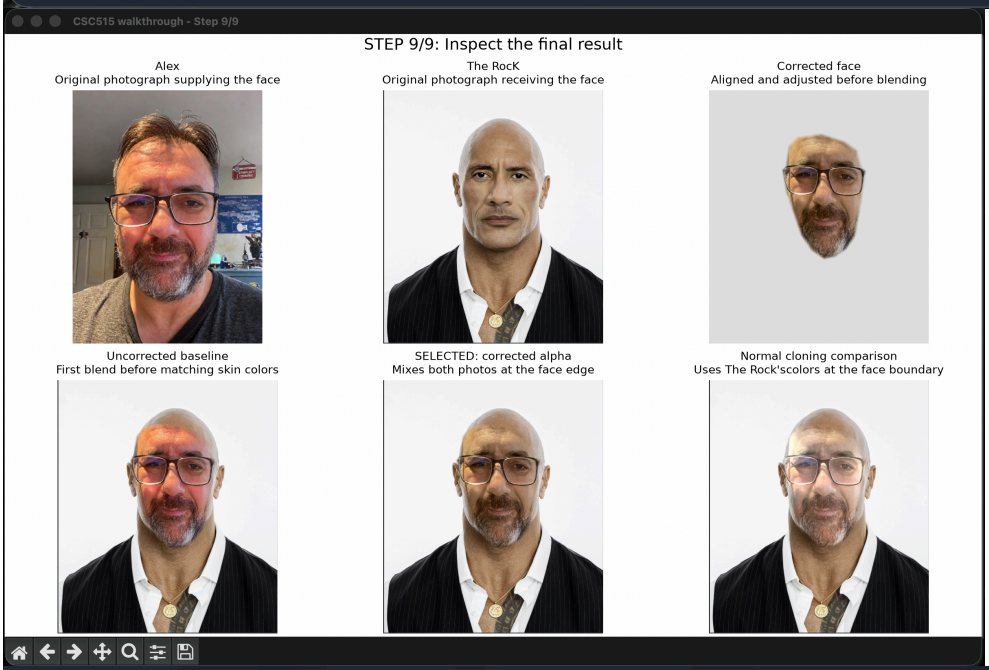
```
Enter C to continue or Q to quit, then press Enter.
c
Opening step 7/9...

================================================================================
STEP 7/9: Blend the corrected face
================================================================================
Mix the corrected face with The Rock using one weight per pixel

Alpha blending combines corrected_alex_face with the_rock_original_image. The center of the selected
face uses Alex, the outside keeps The RocK, and the edge mixes both. Changing the share gradually
makes the replacement meet the surrounding photograph without a sharp pasted edge.

  Code in create_alex_on_the_rock_results():
    alpha_blend_foreground_with_background(corrected_alex_face, the_rock_original_image, face_masks["alex_blend_alpha"])
  Code in alpha_blend_foreground_with_background():
    foreground_weights = foreground_alpha.astype(np.float32)[..., np.newaxis]
    blended_bgr_float = (foreground_weights * foreground_image.astype(np.float32)
              + (1.0 - foreground_weights) * background_image.astype(np.float32))
    blended_bgr_image = np.clip(np.rint(blended_bgr_float), 0, 255).astype(np.uint8)

Inside alpha_blend_foreground_with_background(), foreground_image receives Alex's corrected face and
background_image receives The Rock's photograph. foreground_alpha is the blending mask. Adding
np.newaxis makes each weight apply to blue, green, and red together. float32 keeps fractions during
the calculation. np.rint() rounds the result and np.clip() keeps values between 0 and 255 before
conversion to uint8.

Mix the blue, green, and red values at one pixel:
  weight = foreground_alpha[y, x]
  Alex's share = weight
  The Rock's share = 1 - weight

  blue_result  = weight * blue_alex  + (1 - weight) * blue_the_rock
  green_result = weight * green_alex + (1 - weight) * green_the_rock
  red_result   = weight * red_alex   + (1 - weight) * red_the_rock

Each equation mixes one color channel at the same pixel position in both images. The names ending in
_alex represent values from Alex's corrected face; those ending in _the_rock represent values from
The Rock's original photograph. In foreground_alpha[y, x], y selects the row and x selects the
column. The weight is Alex's share, from 0 to 1. The remaining share comes from The Rock. The same
weight applies to blue, green, and red. It does not move the pixel or exchange channels. The program
then rounds and clips these results to 0–255. Where foreground_alpha is zero, it also copies The
Rock's original pixels directly into the result to keep the background exact.

Example: At foreground_alpha[y, x]=0.5, Alex contributes half and The Rock contributes half. At 1,
the result uses Alex fully. At 0, it keeps The RocK.

Reading the pictures:
Left: Alex's corrected face on gray. Middle: The Rock's original photograph. Right: the
alpha-blended result. Gray is only a preview background. The calculation uses Alex's face colors and
The Rock's original pixels.

Images: (1370, 1192, 3). foreground_alpha: (1370, 1192). foreground_weights: (1370, 1192, 1). The
result returns to uint8 BGR.
Saved teaching view:
/Volumes/P-SSD/Omegapy/CSU-projects/CSC515-CTAs/Portfolio-Milestone-Module-3-Option-2/mod_images/diagnostics/walkthrough/stage_07.png

Enter C to continue or Q to quit, then press Enter.
```



```
Enter C to continue or Q to quit, then press Enter.
c
Opening step 8/9...

================================================================================================
STEP 8/9: Compare with seamless cloning
================================================================================================
Compare a blend based on neighboring color changes and boundary colors

Seamless cloning combines the same corrected face with The Rock in a different way. It uses
gradients, meaning color changes between neighboring face pixels, and fits them to The Rock'scolors
at the selected boundary. This can reduce a seam, but it can also change the brightness of the
replacement face.

  Code in clone_alex_face_into_the_rock():
    alex_clone_x, alex_clone_y, alex_clone_width, alex_clone_height = cv2.boundingRect(alex_face_selection_mask)
    alex_clone_center = (alex_clone_x + alex_clone_width // 2, alex_clone_y + alex_clone_height // 2)
    alex_on_the_rock_clone = cv2.seamlessClone(
        corrected_alex_face, the_rock_original_image, alex_face_selection_mask.copy(), alex_clone_center,
        cv2.NORMAL_CLONE)
    return alpha_blend_foreground_with_background(alex_on_the_rock_clone, the_rock_original_image, alex_blend_alpha), alex_clone_center

boundingRect(alex_face_selection_mask) finds the smallest rectangle around the selected face. It
returns alex_clone_x, alex_clone_y, alex_clone_width, and alex_clone_height. The center of that
rectangle tells seamlessClone() where to place it. NORMAL_CLONE selects the cloning method. The face
was already aligned, so the center keeps it at its existing position.

How the cloning inputs contribute:
  corrected_alex_face -> neighboring color changes -+
  the_rock_original_image -----> colors at the face boundary +-> alex_on_the_rock_clone

  alex_on_the_rock_clone + the_rock_original_image + alex_blend_alpha -> alpha_blend_foreground_with_background() -> final clone result

The arrows show the inputs OpenCV uses to solve for the cloned face colors. It is not a fixed
weighted sum of the two original images. After cloning, alpha_blend_foreground_with_background()
applies the gradual edge and restores the original pixels outside the mask. Its foreground is
alex_on_the_rock_clone here, so this last step keeps the cloning result instead of replacing it with
Alex's pre-cloning colors.

Reading the pictures:
Left: Alex's corrected face. Middle: The Rock with the placement rectangle and center marked. Right:
the cloned result. Compare the forehead, lenses, and beard with the alpha blend from Step 7. Both
methods receive the same corrected face and original photograph.

Both corrected methods produce (1370, 1192, 3) uint8 BGR images. alex_face_selection_mask is a 2D
uint8 selection; alex_clone_center stores an (x, y) position.
Saved teaching view:
/Volumes/P-SSD/Omegapy/CSU-projects/CSC515-CTAs/Portfolio-Milestone-Module-3-Option-2/mod_images/diagnostics/walkthrough/stage_08.png

Enter C to continue or Q to quit, then press Enter.
```



```
Enter C to continue or Q to quit, then press Enter.
c
Opening step 9/9...

================================================================================
STEP 9/9: Inspect the final result
================================================================================
Inspect the comparison and the saved face replacement

The selected result uses skin-color matching followed by alpha blending. Normal cloning made the
forehead and lenses too bright in this comparison. The alpha result keeps more of that detail while
reducing the red tint. The earlier baseline remains visible so the effect of the changes is easier
to judge.

  Code in main():
    save_bgr_image_as_png(
        face_blending_results["alex_on_the_rock_candidates"][
            SELECTED_ALEX_ON_THE_ROCK_METHOD],
        ALEX_ON_THE_ROCK_RESULT_PATH)
    save_alex_the_rock_comparison(face_blending_results)

The band of light on Alex's forehead and the different head and chin shapes remain visible. Matching
skin colors does not recreate the lighting or the three-dimensional shape of a face. This milestone
uses manually chosen points. Automatic face detection and eye blurring belong to the later portfolio
project.

The comparison paths from the same two photographs:
  aligned_alex_face   + alex_baseline_alpha -> alex_on_the_rock_baseline
  corrected_alex_face + face_masks["alex_blend_alpha"] -> alex_on_the_rock_alpha (selected)
  corrected_alex_face + face_masks["alex_face_selection_mask"]  -> alex_on_the_rock_clone

  Each path also uses the_rock_original_image.

The baseline uses its original Gaussian mask. Both corrected methods use the same color-adjusted
face; the clone also receives the final alpha edge. SELECTED_ALEX_ON_THE_ROCK_METHOD chooses the
saved replacement. ALEX_ON_THE_ROCK_RESULT_PATH holds that image, and ALEX_THE_ROCK_COMPARISON_PATH
holds the four-panel comparison. Saving these derived files leaves both photographs in org_images
unchanged.

Reading the pictures:
Top row: Alex, The RocK, and the corrected face on gray. Bottom row: the first blend before color
matching, the selected alpha blend, and the seamless-cloning comparison. Check the skin color, the
face boundary, and the detail around the glasses and beard. The surrounding photograph should stay
the same.

Saved replacement: (1370, 1192, 3), uint8 BGR. Display panels convert a copy to RGB; they do not
change the saved face-replacement pixels.
Saved teaching view:
/Volumes/P-SSD/Omegapy/CSU-projects/CSC515-CTAs/Portfolio-Milestone-Module-3-Option-2/mod_images/diagnostics/walkthrough/stage_09.png

Enter Q to quit, then press Enter.
```




```
Enter Q to quit, then press Enter.
q
The result and comparison are saved in mod_images/.
alex.omegapy@MacStudio CSC515-CTAs %
```